

LABSHEET 4

DISHA SAMANTA

CU24250227

BTECH CSE B

ROLLNO 20

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
from google.colab import files

uploaded = files.upload()

filename = next(iter(uploaded))

img = cv2.imread(filename, cv2.IMREAD_GRAYSCALE)

if img is None:
    print("Error: Image could not be loaded.")
else:
    print("Image loaded successfully!")
    print("Image shape:", img.shape)

plt.figure(figsize=(7,5))
plt.imshow(img, cmap='gray')
plt.title("Original Grayscale Image")
plt.axis("off")
plt.show()
```

Choose files .jpeg
.jpeg(image/jpeg) - 54457 bytes, last modified: 03/09/2026 - 100% done
Saving .jpeg to  (1).jpeg
Image loaded successfully!
Image shape: (717, 736)

Original Grayscale Image



```
# Convert image to float32
img_float = np.float32(img)

# Compute DFT
dft = cv2.dft(img_float, flags=cv2.DFT_COMPLEX_OUTPUT)

print("DFT calculated successfully!")
print("DFT Shape:", dft.shape)
```

DFT calculated successfully!
DFT Shape: (717, 736, 2)

```
# Shift zero-frequency component to the center
dft_shift = np.fft.fftshift(dft)

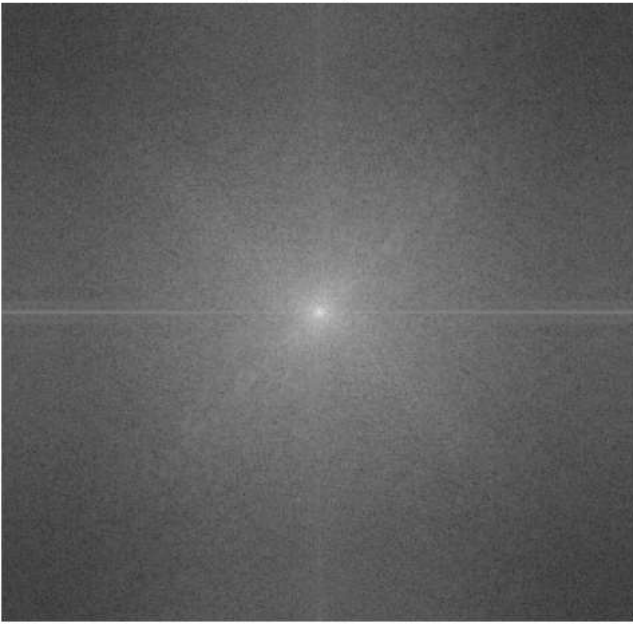
# Calculate magnitude
magnitude = cv2.magnitude(
    dft_shift[:, :, 0],
    dft_shift[:, :, 1]
)

# Log transformation for better visualization
```

```
magnitude_spectrum = 20 * np.log(magnitude + 1)
```

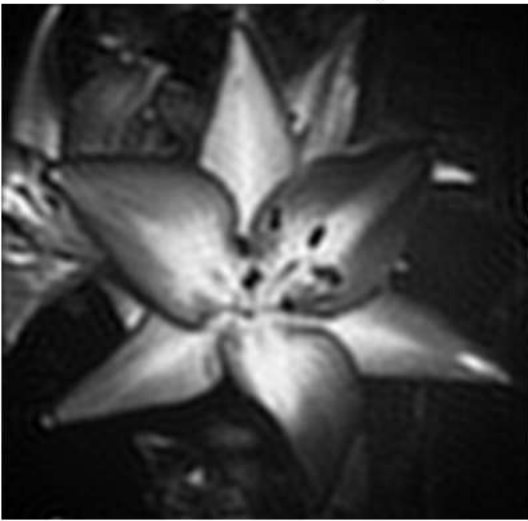
```
plt.figure(figsize=(8,6))  
plt.imshow(magnitude_spectrum, cmap='gray')  
plt.title("Magnitude Spectrum")  
plt.axis("off")  
plt.show()
```

Magnitude Spectrum



```
# Get image dimensions  
rows, cols = img.shape  
  
# Center coordinates  
crow = rows // 2  
ccol = cols // 2  
  
# Radius of Low-Pass Filter  
radius = 40  
  
# Create coordinate grid  
Y, X = np.ogrid[:rows, :cols]  
  
# Calculate distance from center  
distance = np.sqrt((X - ccol)**2 + (Y - crow)**2)  
  
# Create Low-Pass Filter mask  
low_pass_mask = np.zeros((rows, cols, 2), np.float32)  
  
low_pass_mask[distance <= radius] = 1  
  
# Apply Low-Pass Filter  
low_pass_dft = dft_shift * low_pass_mask  
  
# Shift back  
low_pass_ishift = np.fft.ifftshift(low_pass_dft)  
  
# Inverse Fourier Transform  
low_pass_result = cv2.idft(low_pass_ishift)  
  
# Calculate magnitude  
low_pass_result = cv2.magnitude(  
    low_pass_result[:, :, 0],  
    low_pass_result[:, :, 1]  
)  
  
# Normalize  
low_pass_result = cv2.normalize(  
    low_pass_result,  
    None,  
    0,  
    255,  
    cv2.NORM_MINMAX  
) .astype(np.uint8)  
  
# Display  
plt.figure(figsize=(7,5))  
plt.imshow(low_pass_result, cmap='gray')  
plt.title("Low-Pass Filtered Image")  
plt.axis("off")  
plt.show()
```

Low-Pass Filtered Image



```
# Create High-Pass Filter mask
high_pass_mask = np.ones((rows, cols, 2), np.float32)

high_pass_mask[distance <= radius] = 0

# Apply High-Pass Filter
high_pass_dft = dft_shift * high_pass_mask

# Shift back
high_pass_ishift = np.fft.ifftshift(high_pass_dft)

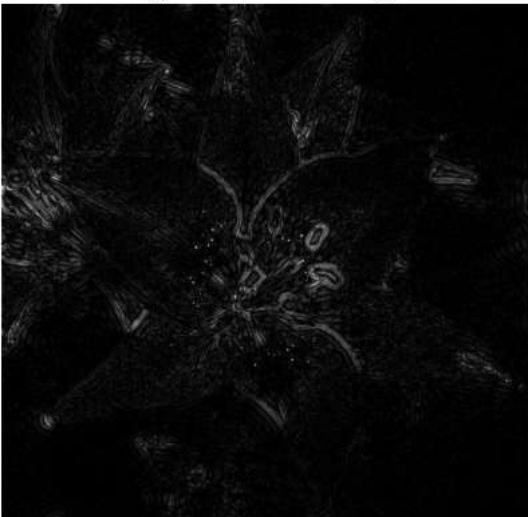
# Inverse Fourier Transform
high_pass_result = cv2.idft(high_pass_ishift)

# Calculate magnitude
high_pass_result = cv2.magnitude(
    high_pass_result[:, :, 0],
    high_pass_result[:, :, 1]
)

# Normalize
high_pass_result = cv2.normalize(
    high_pass_result,
    None,
    0,
    255,
    cv2.NORM_MINMAX
).astype(np.uint8)

# Display
plt.figure(figsize=(7,5))
plt.imshow(high_pass_result, cmap='gray')
plt.title("High-Pass Filtered Image")
plt.axis("off")
plt.show()
```

High-Pass Filtered Image



```
plt.figure(figsize=(14,10))

plt.subplot(2,2,1)
```

```
plt.imshow(img, cmap='gray')
plt.title("Original Image")
plt.axis("off")

plt.subplot(2,2,2)
plt.imshow(magnitude_spectrum, cmap='gray')
plt.title("Magnitude Spectrum")
plt.axis("off")

plt.subplot(2,2,3)
plt.imshow(low_pass_result, cmap='gray')
plt.title("Low-Pass Filter")
plt.axis("off")

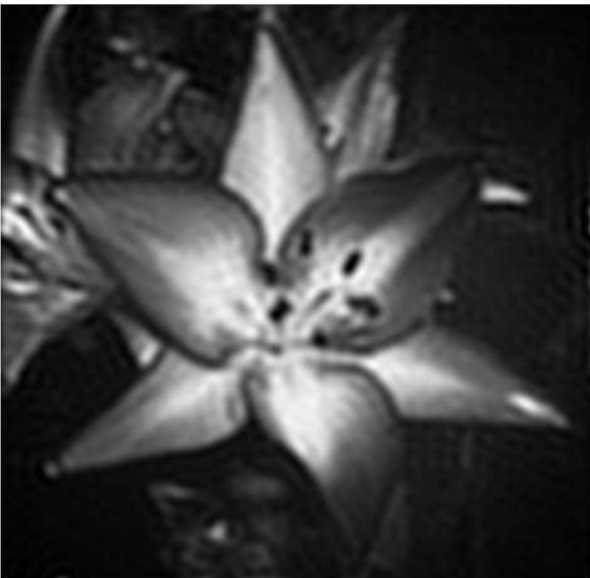
plt.subplot(2,2,4)
plt.imshow(high_pass_result, cmap='gray')
plt.title("High-Pass Filter")
plt.axis("off")

plt.tight_layout()
plt.show()
```

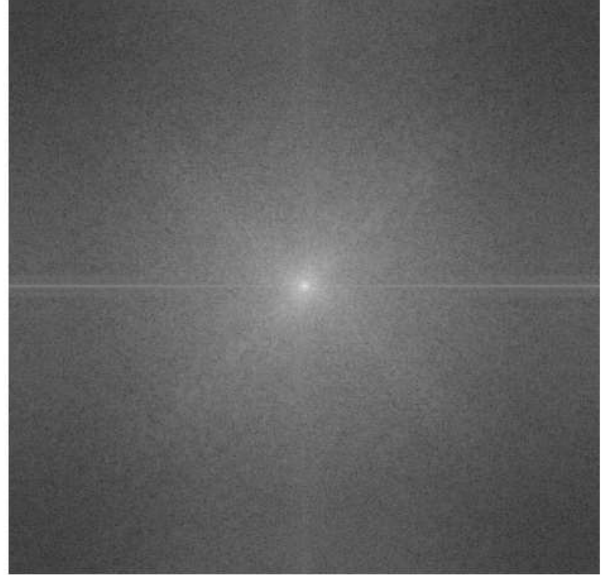
Original Image



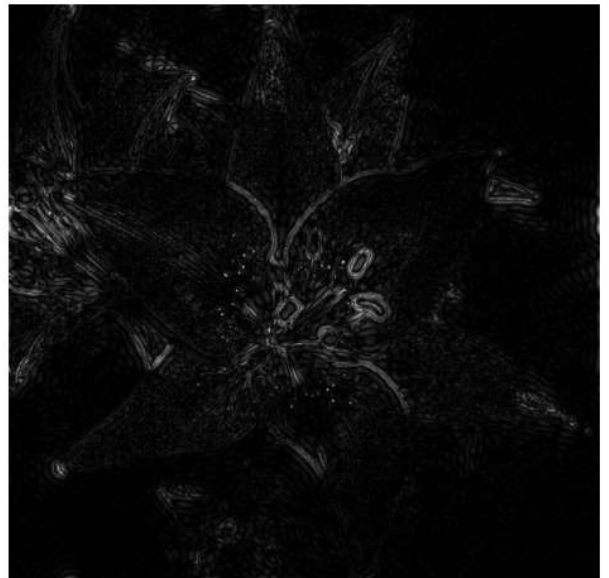
Low-Pass Filter



Magnitude Spectrum



High-Pass Filter



```
print("RESULT ANALYSIS")
print("=" * 50)

print("1. Original Image:")
print("    Contains all spatial information and image details.")

print("\n2. Magnitude Spectrum:")
print("    Shows the distribution of frequency components.")

print("\n3. Low-Pass Filter:")
print("    Reduces high-frequency components, noise and fine details.")
print("    Produces a smoother image.")
```

```
print("\n4. High-Pass Filter:")
print("    Removes low-frequency components and enhances edges and fine details.")
```

RESULT ANALYSIS

=====

1. Original Image:
 Contains all spatial information and image details.

2. Magnitude Spectrum:
 Shows the distribution of frequency components.

3. Low-Pass Filter:
 Reduces high-frequency components, noise and fine details.
 Produces a smoother image.

4. High-Pass Filter:
 Removes low-frequency components and enhances edges and fine details.